M1PPAW - Projets Cybersécurité Avancés

🎯 PROJET FIL ROUGE : Application Web Sécurisée

Concept : Application web durcie contre les vulnérabilités OWASP Top 10.

Durée : 5 jours

Jour 1 : Setup Environnement et Analyse Préliminaire

Semaine 10 - M1 Cybersécurité | 7 heures

★ Contextualisation Professionnelle

A. Pourquoi ce module ?

Problème résolu : Les entreprises ont besoin de professionnels capables de mener des missions complètes et autonomes : audits de sécurité, analyse d'incidents, reverse engineering de menaces. Ce module développe la capacité à gérer un projet cybersécurité de A à Z.

- **75%** des RSSI estiment manquer de talents capables de mener des missions bout-en-bout
- **3000€/jour** : tarif moyen d'un consultant senior en pentest/forensics
- **+60%** de demande pour des profils "Purple Team" (attaque + défense)

- Les projets complets sont la meilleure préparation aux certifications OSCP, GREM, GWAPT

B. Métiers concernés

Métier	Salaire débutant	Salaire confirmé	Évolution
Pentester / Ethical Hacker	40-48K€	60-90K€	Lead Pentester, Red Team Manager
Analyste Malware / Reverse Engineer	45-55K€	70-100K€	Head of Threat Intelligence
Consultant Cybersécurité Senior	45-55K€	80-120K€	Directeur Conseil, CISO
Security Researcher	50-60K€	80-130K€	Principal Researcher, Bug Bounty Hunter
Purple Team Specialist	48-58K€	75-110K€	Head of Security Operations

C. Liens avec le programme École IT

Prérequis :

- M1CRYP - Cryptographie (chiffrement, PKI)
- M1PENT - Penetration Testing (OWASP, méthodologies)
- B3 Réseaux & Sécurité (firewall, IDS)
- Python avancé (scripting, automation)

Complémente :

- M1 DevSecOps (intégration sécurité)
- M1 Architecture sécurisée

- M1 Forensics (analyse post-incident)

Prépare à :

- M2 Red Team Operations
- M2 Threat Intelligence
- Certifications : OSCP, GREM, GWAPT, eCPPT

D. Types de projets en entreprise

- **Audit de sécurité complet** - Pentest black/white box pour clients
- **Analyse de malware** - Reverse engineering pour SOC/CERT
- **Exercices Red Team** - Simulations d'attaques APT
- **Bug Bounty Programs** - Chasse aux vulnérabilités rémunérée
- **Response à incident** - Analyse forensique post-compromission
- **Audit sécurité mobile** - Test d'applications banking/fintech
- **Purple Team exercises** - Amélioration continue SOC
- **Threat hunting** - Recherche proactive de menaces

E. Technologies et outils

Ghidra IDA Pro Burp Suite Metasploit YARA Sigma Frida jadx **Kali Linux** **REMnux** **FlareVM** **ELK Stack** **Suricata** Python scripting Wireshark VirusTotal MalwareBazaar

F. Entreprises qui recrutent

Cabinets de conseil : Wavestone, Orange Cyberdefense, Deloitte Cyber, KPMG, NCC Group, Mandiant
Éditeurs sécurité : CrowdStrike, SentinelOne, Kaspersky, ESET, Recorded Future
Bug Bounty : HackerOne, Bugcrowd, YesWeHack, Intigriti

Grands groupes : Airbus CyberSecurity, Thales, Capgemini, Atos, Sopra Steria
Banques/Finance : BNP Paribas, Société Générale, AXA (équipes Red Team internes)

G. Domaines d'application

Finance & Banque | **Défense & Gouvernement** | **Santé & Pharmaceutique** | **Énergie & Industrie** | **Télécommunications** | **E-commerce** | **Conseil & Audit** | **Assurance** | **Gaming & Divertissement** | **IoT & Systèmes embarqués**

H. Combinaisons technologiques en entreprise

Stack	Combinaison	Cas d'usage entreprise
SOC Automation	SIEM + SOAR + Python + Threat Intel	Automatisation réponse incidents, enrichissement alertes
Purple Team	Red Team tools + Blue Team + MITRE ATT&CK	Amélioration continue défense, tests d'attaque réalistes
DevSecOps	CI/CD + SAST/DAST + Container Security + IaC Scan	Sécurité intégrée dans pipelines, shift-left security
Forensics & IR	Volatility + KAPE + Velociraptor + Timeline	Investigation incidents, collecte de preuves, analyse mémoire
Threat Hunting	SIEM + ELK + YARA + Sigma Rules	Détection proactive menaces, hunting hypothèses

Dans votre formation École IT : Ce projet (M1PPAW) intègre cryptographie (M1CRYP) et pentest (M1PENT). En M2, vous appliquerez ces compétences à la sécurité IoT/OT (M2SECM) et à l'automatisation Python pour la cybersécurité (M2PYTS).

Objectifs pédagogiques du jour

- Choisir et justifier un projet parmi les 3 options proposées
- Déployer un environnement d'analyse sécurisé et isolé
- Effectuer une reconnaissance initiale avec méthodologie professionnelle
- Élaborer un plan d'attaque structuré avec timeline et livrables
- Documenter les premières découvertes selon standards de l'industrie

Contexte Professionnel

Situation : Vous êtes consultant senior en cybersécurité pour une société de conseil (Deloitte, KPMG, Wavestone). Un client vous confie une mission d'audit/investigation de 5 jours. Les 8 premières heures sont critiques : mauvais choix d'outils ou méthodologie bâclée = échec de mission.

Enjeux réels :

- Budget client : 15 000€ pour 5 jours (3000€/jour)
- Réputation de votre cabinet en jeu
- Rapport exécutif attendu pour COMEX/RSSI
- Conformité RGPD/ISO27001 obligatoire

MATINÉE : Choix du Projet et Setup Environnement ()

1. Présentation des 3 Projets Disponibles

PROJET 1 : Reverse Engineering Malware Bancaire

Objectif : Analyser un échantillon de malware bancaire (famille ZeuS/Emotet/TrickBot) pour extraire les IOCs, comprendre les mécanismes de persistance et proposer des contre-mesures.

Compétences mobilisées :

- Analyse statique (IDA Pro/Ghidra, strings, PE headers)
- Analyse dynamique (sandbox, API monitoring, network capture)
- Reverse engineering (désassemblage x86/x64, déobfuscation)
- Scripting Python pour automatisation
- Création de règles YARA pour détection

Profil recommandé : Passion pour le bas niveau, curiosité technique, patience pour l'analyse de binaires

Cas d'usage professionnel : SOC/CERT en banque, assurance, éditeur d'antivirus

PROJET 2 : Red Team vs Blue Team - E-commerce

Objectif : Auditer l'application web "SecureShop" (vulnérabilités intentionnelles), l'exploiter en mode Red Team, puis mettre en place des défenses en mode Blue Team.

Compétences mobilisées :

- Pentest Web (XSS, SQLi, CSRF, IDOR, file upload)
- Exploitation avancée (chaining, privilege escalation)
- Défense active (WAF, IDS/IPS, SIEM)
- Analyse de logs avec ELK Stack
- Threat hunting et forensics

Profil recommandé : Expérience en pentest web, envie de comprendre les deux côtés (attaque/défense)

Cas d'usage professionnel : ESN de pentest, équipe Purple Team en entreprise

PROJET 3 : Audit Sécurité Mobile Banking

Objectif : Auditer l'APK "MobiBank" (application bancaire fictive mais réaliste), identifier les vulnérabilités de sécurité, tester la sécurité des communications et du stockage des données.

Compétences mobilisées :

- Reverse engineering Android (jadx, apktool, dex2jar)
- Analyse dynamique avec Frida (hooking, instrumentation)
- Test de sécurité API REST/HTTPS
- Analyse stockage sensible (SharedPreferences, SQLite, Keystore)
- Exploitation (root detection bypass, certificate pinning bypass)

Profil recommandé : Intérêt pour la sécurité mobile, connaissances Java/Kotlin, curiosité pour l'écosystème Android

Cas d'usage professionnel : RSSI banque/fintech, auditeur mobile, éditeur d'applications sensibles

TP1 : Choix et Justification de Projet (45 min)

Contexte : Votre N+1 vous demande de choisir une mission parmi les 3 proposées par des clients. Vous devez justifier votre choix par un document d'1 page.

Livrables attendus :

1. Document `choix_projet.md` contenant :
    - Projet choisi (1, 2 ou 3)
    - 3 arguments techniques justifiant le choix

- 2 risques identifiés et stratégies de mitigation
- Liste des compétences mobilisées et lacunes éventuelles
- Estimation macro du planning (% de temps par phase)

Critères d'évaluation professionnelle :

- Honnêteté sur vos compétences actuelles (vs survente dangereuse)
- Réalisme de l'estimation temporelle
- Anticipation des blocages techniques
- Alignement avec votre projet de carrière

> **Conseil Pro :** En entreprise, choisir une mission inadaptée à vos compétences peut coûter très cher (client perdu, réputation entamée). Il vaut mieux décliner une mission ou demander du renfort que d'échouer. Mais c'est aussi l'occasion de sortir de sa zone de confort avec un filet de sécurité (formation).

2. Setup Environnement Sécurisé

> **SÉCURITÉ CRITIQUE**
> L'analyse de malware ou l'exploitation de vulnérabilités DOIT se faire dans un environnement isolé. Une erreur de manipulation peut compromettre votre machine hôte, le réseau EcoleIT, ou violer des lois (RGPD, LPM).

2.1. Architecture d'Isolation Recommandée

Standard Industrie : Multi-Layer Isolation

```
┌─────────────────────────────────────────────────────────
│ Machine Hôte (macOS/Windows/Linux) │
│ ┌──────────────────────────────────────────────┐ │
│ │ Hyperviseur (VMware/VirtualBox/KVM) │ │
│ │ ┌──────────────────────────────────────┐ │ │
│ │ │ VM d'Analyse (Kali/REMnux/FlareVM) │ │ │
```

```
│ │ │ - Network: Host-Only ou NAT isolé │ │ │
│ │ │ - Snapshots avant chaque manipulation │ │ │
│ │ │ - Shared folders désactivés │ │ │
│ │ └──────────────────────────────────────┘ │ │
│ └──────────────────────────────────────────┘ │
```

Pourquoi cette architecture ?

- Snapshot = retour arrière instantané si compromission
- Host-Only = aucune communication avec Internet/LAN
- Hyperviseur moderne = isolation CPU/mémoire robuste

2.2. Configurations Spécifiques par Projet

Configuration Projet 1 - Reverse Malware

```
# VM recommandée : REMnux (distribution dédiée au reverse)
# Téléchargement : https://remnux.org/ # Outils essentiels à
sudo apt update && sudo apt install -y \ ghidra \ radare2 \
wget https://out7.hex-rays.com/files/idafree82_linux.run
chmod +x idafree82_linux.run
./idafree82_linux.run # Python libraries pour analyse
pip3 install pefile yara-python capstone unicorn pycryptodom
# Vérification isolation
ping -c 1 8.8.8.8 # Doit échouer
```

> **Astuce Pro :** Les analystes malware professionnels utilisent souvent une VM par échantillon analysé. Coût disque faible (20-30 GB par VM), sécurité maximale. Utilisez des templates clonables.

Configuration Projet 2 - Red/Blue Team

```
# VM recommandée : Kali Linux 2024.x
# Téléchargement : https://www.kali.org/get-kali/ # Outils F
sudo apt update && sudo apt install -y \ burpsuite \ sqlmap
sudo apt install -y \ elasticsearch \ logstash \ kibana \ su
docker pull docker.elastic.co/elasticsearch/elasticsearch:8.
```

```
docker pull docker.elastic.co/logstash/logstash:8.11.0
docker pull docker.elastic.co/kibana/kibana:8.11.0 # Déploie
git clone https://github.com/ecoleit-cyber/secureshop-vulner
cd secureshop-vulnerable
```

Configuration Projet 3 - Mobile Banking

```
# VM recommandée : Ubuntu 22.04 LTS avec Android Studio
# + Genymotion ou émulateur Android standard # Installation
sudo apt install -y android-sdk adb # Outils Android Securit
sudo apt install -y \ apktool \ dex2jar \ jd-gui # Installat
wget https://github.com/skylot/jadx/releases/download/v1.4.7
unzip jadx-1.4.7.zip -d /opt/jadx
export PATH=$PATH:/opt/jadx/bin # Installation Frida
pip3 install frida-tools
pip3 install objection # Configuration émulateur Android (AF
avdmanager create avd -n "SecurityTest" -k "system-images;ar
emulator -avd SecurityTest -no-snapshot-load & # Installatic
adb root
adb push burp_cert.der /sdcard/
adb shell settings put global verifier_verify_adb_installs 0
```

TP2 : Déploiement et Validation Environnement ()

Objectif : Installer et valider un environnement opérationnel pour votre projet choisi.

Checklist de validation (à documenter avec screenshots) :

Pour TOUS les projets :

1. VM créée avec ressources adaptées (RAM ≥ 4GB, CPU ≥ 2 cores)
2. Snapshot "Clean Install" créé et testé (restauration OK)
3. Réseau configuré en Host-Only (test ping Internet = échec)
4. Outils principaux installés et testés (lancer chaque outil une fois)
5. Dossier projet structuré :

```
~/projet_m1ppaw/
├── docs/ # Documentation et notes
├── tools/ # Scripts et outils custom
├── evidence/ # Preuves (screenshots, pcaps, logs)
├── reports/ # Rapports intermédiaires et final
└── artifacts/ # Échantillons, APK, dumps mémoire
```

Validation spécifique Projet 1 (Malware) :

- Ghidra lance et charge un binaire de test (`/bin/ls`)
- YARA détecte une règle de test
- Script Python lit un PE header (test sur `calc.exe`)

Validation spécifique Projet 2 (Red/Blue) :

- SecureShop accessible via navigateur (http://localhost:8080)
- Burp Suite intercepte le trafic HTTP de l'app
- ELK Stack démarré (Kibana accessible sur localhost:5601)

Validation spécifique Projet 3 (Mobile) :

- Émulateur Android boot complet
- APK MobiBank installé (`adb install mobibank.apk`)
- Frida se connecte à l'app : `frida-ps -U | grep mobibank`
- jadx décompile l'APK sans erreur

Livrables :

- Document `setup_validation.md` avec tous les checkpoints cochés
- Screenshots dans `evidence/setup/` prouvant chaque étape
- Script de validation automatique (bonus) testant les 5 points génériques

> **BLOCAGE FRÉQUENT :** L'installation d'outils prend souvent plus de temps que prévu (dépendances manquantes, compatibilité OS). Prévoyez large et testez AVANT de manipuler des données sensibles.

💡 **Guide de dépannage par problème :**

- **Ghidra ne lance pas :** Vérifier Java JDK 11+ avec `java -version` . Installer avec `sudo apt install openjdk-17-jdk`
- **Burp Suite erreurs SSL :** Exporter le certificat CA de Burp (Proxy > Options > Import/Export CA cert), puis l'installer dans Firefox
- **Frida "Failed to spawn" :** Vérifier versions identiques client/server avec `frida --version` . Réinstaller si différentes
- **Docker ELK Stack crash :** RAM insuffisante. Augmenter à 6GB minimum. Vérifier avec `docker stats`
- **APK ne s'installe pas :** Activer "Unknown sources" avec `adb shell settings put global install_non_market_apps 1`

3. Première Reconnaissance

Méthodologie Professionnelle : OSINT Before Tools

Avant de lancer Ghidra/Burp/Frida, un consultant senior passe 30-60 minutes à collecter des informations passives :

- Recherche de CVE similaires (exploit-db, nvd.nist.gov)
- Lecture de rapports d'incidents publics (blogs de SOC/CERT)
- Analyse de malware similaires (MalwareBazaar, VirusTotal)
- Veille technique sur les techniques d'exploitation récentes

Gain : Éviter de perdre 2 jours sur un problème déjà résolu ailleurs. Identifier les pièges classiques.

3.1. Reconnaissance Projet 1 - Malware

```
# Informations de base sur le binaire (SANS L'EXÉCUTER)
file malware_sample.exe
# Sortie attendue : PE32 executable (GUI) Intel 80386, for M
sha256sum malware_sample.exe
md5sum malware_sample.exe # Strings basiques (indice sur for
```

```
strings malware_sample.exe | grep -E "(http|ftp|cmd|powershe
python3 << 'EOF'
import pefile pe = pefile.PE('malware_sample.exe')
print(f"[+] Compilation timestamp : {pe.FILE_HEADER.TimeDate
print(f"[+] Sections :")
for section in pe.sections: print(f" {section.Name.decode().
for entry in pe.DIRECTORY_ENTRY_IMPORT: print(f" DLL: {entry
EOF # Détection basique avec YARA (règles génériques)
yara -r /opt/yara-rules/malware/ malware_sample.exe
```

Indicateurs de Malware Classiques :

- Sections avec noms anormaux (`.sexy` , `.packed`)
- Ratio Virtual Size / Raw Size > 2 → probable packing/compression
- Imports suspects : `VirtualAllocEx` , `WriteProcessMemory` , `CreateRemoteThread` → injection de code
- Timestamp incohérent (futur ou trop ancien)

3.2. Reconnaissance Projet 2 - Web App

```
# Énumération manuelle de l'app
firefox http://localhost:8080 & # Crawling et cartographie
gobuster dir -u http://localhost:8080 -w /usr/share/seclists
whatweb http://localhost:8080 # Analyse robots.txt et sitema
curl http://localhost:8080/robots.txt
curl http://localhost:8080/sitemap.xml # Scan de vulnérabili
nikto -h http://localhost:8080 -output nikto_baseline.html -
sslscan localhost:8443 # Capture du trafic pour analyse ulté
sudo tcpdump -i lo port 8080 -w baseline_traffic.pcap &
# Naviguer manuellement 5 minutes sur l'app, puis :
sudo pkill tcpdump
```

3.3. Reconnaissance Projet 3 - Mobile APK

```
# Informations de base sur l'APK
aapt dump badging mobibank.apk | grep -E "(package|sdkVersic
apktool d mobibank.apk -o mobibank_decompiled
```

```
cat mobibank_decompiled/AndroidManifest.xml | grep -E "(perm
jadx -d mobibank_jadx mobibank.apk # Recherche de secrets ha
grep -ri "password\|api_key\|secret\|token" mobibank_jadx/sc
jarsigner -verify -verbose -certs mobibank.apk # Extraction
strings mobibank.apk | grep -E "https?://" > endpoints.txt #
```

APRÈS-MIDI : Plan d'Attaque et Documentation Initiale ()

4. Élaboration du Plan d'Attaque

Kill Chain Adapté à Chaque Projet

En cybersécurité offensive, on utilise souvent le modèle "Cyber Kill Chain" de Lockheed Martin. Pour nos projets :

- **Projet 1 (Malware) :** Analyse statique → Analyse dynamique → Reverse engineering → IOC extraction → Mitigation
- **Projet 2 (Web) :** Reconnaissance → Scanning → Exploitation → Post-exploitation → Défense
- **Projet 3 (Mobile) :** Recon statique → Recon dynamique → Exploitation → Bypass protections → Remediation

TP3 : Rédaction du Plan d'Attaque (2h)

Objectif : Produire un document de planification professionnel type "Statement of Work" (SOW).

Structure obligatoire du document `plan_attaque.md` **:**

1. Executive Summary (1 paragraphe)

- Projet choisi et objectif global
- Méthodologie principale appliquée
- Livrables finaux attendus

2. Scope et Périmètre

- Assets analysés (binaire/app web/APK)
- Ce qui est IN SCOPE : quels tests sont autorisés
- Ce qui est OUT OF SCOPE : lignes rouges (ex : pas de DoS)
- Hypothèses et contraintes (temps, outils disponibles)

3. Méthodologie Détaillée

Décomposer en phases avec pour chacune :

- Objectifs spécifiques
- Outils utilisés
- Critères de succès
- Temps estimé
- Risques/blocages potentiels

Exemple pour Projet 1 (Malware) :

```
### Phase 1 : Analyse Statique Approfondie (Jour 2, 4h
**Objectifs :**
- Identifier les fonctions critiques (C2, persistence,
- Détecter les techniques d'obfuscation/packing
- Extraire les strings décodées **Outils :**
- Ghidra pour désassemblage
- CFF Explorer pour analyse PE
- PEiD pour détection de packer
- Script Python custom pour déobfuscation XOR **Critèr
- Graphe de flux de contrôle complet dans Ghidra
- Liste des API Windows critiques identifiées
- Au moins 3 strings décodées manuellement **Temps est
- Packer custom inconnu → mitigation : utiliser unpack
- Obfuscation lourde → mitigation : analyse dynamique
```

Exemple pour Projet 2 (Web - Red/Blue Team) :

```
### Phase 1 : Audit OWASP Top 10 (Jour 2, 4h)
**Objectifs :**
- Tester systématiquement les 10 vulnérabilités OWASP
- Identifier minimum 5 vulnérabilités exploitables
- Documenter chaque vulnérabilité avec PoC
```

```
**Outils :**
- Burp Suite Pro pour interception et fuzzing
- SQLMap pour injection SQL automatisée
- Nikto pour scan baseline
- Scripts Python custom pour tests CSRF/IDOR

**Critères de succès :**
- Au moins 1 SQLi fonctionnelle avec extraction de dor
- Au moins 2 XSS (stored ou reflected) avec session h:
- CSRF validé sur action sensible (changement email/pa
- 1 IDOR permettant accès à données d'autres utilisate

**Temps estimé :** 4h

**Risques :**
- WAF bloquant les tests → mitigation : bypass WAF ave
```

Exemple pour Projet 3 (Mobile Banking) :

```
### Phase 1 : Analyse Statique et Secrets (Jour 2, 4h)
**Objectifs :**
- Décompiler APK et analyser code source Java/Kotlin
- Identifier secrets hardcodés (API keys, tokens, pass
- Mapper les endpoints API et mécanismes d'authentific
- Analyser la sécurité du stockage local

**Outils :**
- jadx pour décompilation
- apktool pour extraction ressources
- grep/ripgrep pour recherche de patterns
- SQLite browser pour analyse BDD

**Critères de succès :**
- Code source complet décompilé (100% des classes lis:
- Au moins 1 secret hardcodé identifié (API key, toker
- Liste complète des endpoints API avec paramètres
- Schéma de la base SQLite locale documenté

**Temps estimé :** 4h

**Risques :**
- Obfuscation ProGuard/R8 → mitigation : analyse dynar
```

```
  - Certificate pinning empêche interception → mitigatic
```

4. Planning Prévisionnel

Tableau de répartition du temps :

```
| Jour | Phase | Activités Principales | Livrables |
|------|-------|----------------------|-----------|
| J1 | Setup + Recon | Installation, première analyse
| J2 | Analyse approfondie | [Détails selon projet] |
| J3 | Exploitation/Dev outils | [Détails selon projet
| J4 | Approfondissement | [Détails selon projet] | Co
| J5 | Consolidation | Documentation finale, présentat
```

5. Livrables et Formats

- Rapport technique (format, nombre de pages estimé)
- Rapport exécutif (1 page pour décideurs)
- Artefacts techniques (scripts, règles YARA, exploits PoC)
- Présentation (slides pour démonstration)

6. Critères de Qualité

- Reproductibilité des tests (scripts fournis)
- Conformité aux standards (OWASP, PTES, NIST)
- Niveau de détail technique (pour équipe technique)
- Clarté pour décideurs (rapport exécutif)

> **Réalisme :** En entreprise, ce document est validé par le client AVANT de commencer les tests. Un bon SOW protège les deux parties (client sait ce qu'il aura, consultant protégé juridiquement).

5. Documentation des Premières Découvertes

5.1. Standards de Documentation en Cybersécurité

Les 3 Niveaux de Documentation Professionnelle

1. **Notes techniques (daily log) :** Markdown brut, chronologique, pour vous-même. Tout documenter, même les échecs.
2. **Rapport technique :** Structuré, pour pairs techniques (équipe SOC, dev). Format OWASP/PTES.
3. **Rapport exécutif :** 1-2 pages, business language, pour COMEX/RSSI. Focus sur risque métier et coûts.

5.2. Template Notes Techniques (Daily Log)

```
# Daily Log - M1PPAW Projet [N] - Jour 1
**Date :** 2025-XX-XX
**Durée :** 7h
**Objectif du jour :** Setup environnement + première reconn
- **OS :** REMnux 7.x / Kali 2024.1 / Ubuntu 22.04
- **RAM :** 4GB
- **CPU :** 2 cores
- **Network :** Host-Only (vérifié avec `ping 8.8.8.8`)
- **Snapshot :** "Clean_Install" créé à ### Outils Installés
- [x] Ghidra 10.4 (testé avec /bin/ls)
- [x] YARA 4.3.2 (règle test détectée)
- [x] Python 3.11 + pefile + yara-python
- [ ] IDA Free (échec installation → utiliser Ghidra uniquem
**Solution :** Utiliser Ghidra (équivalent open-source) ---
```
MD5 : a3f1b23c4d5e6f7890abcdef12345678
SHA256 : abc123... (complet dans evidence/hashes.txt)
VirusTotal : 35/70 détections → famille probable TrickBot
``` ### Analyse Strings
**Strings suspects identifiés :**
- `http://185.234.xxx.xxx/gate.php` → Serveur C2 potentiel
- `SOFTWARE\Microsoft\Windows\CurrentVersion\Run` → Persiste
- `POST /upload` → Exfiltration de données ? **Actions à pre
- Analyser la fonction qui construit l'URL C2 (probable XOR
- Vérifier si gate.php est encore en ligne (VirusTotal passi
- `.text` : Virtual Size 45KB / Raw Size 20KB → Probable unp
- `.rsrc` : 2MB → Données/DLL embarquées ? **Imports critiqu
- `kernel32.dll` : VirtualAllocEx, WriteProcessMemory → Inje
```

```
- `ws2_32.dll` : socket, connect, send → Communication résea
- `advapi32.dll` : RegSetValueEx → Persistence registre ---
2. Unpacking manuel si nécessaire (dump process memory)
3. Analyse dynamique : exécution en sandbox avec procmon + t
4. Extraction IOCs complets (IP, domaines, mutexes, registry
- Première reconnaissance : - Plan d'attaque : 2h
- Documentation : 1h **Total :** 7h
```

TP4 : Documentation des Découvertes Initiales (1h)

Objectif : Rédiger votre premier daily log professionnel basé sur les TPs 1-3.

Consignes :

1. Créer `docs/daily_logs/jour1.md` selon le template ci-dessus
2. Y inclure TOUTES les commandes exécutées (copier-coller depuis terminal)
3. Pour chaque découverte, ajouter un screenshot dans `evidence/jour1/`
4. Identifier au moins 3 "next steps" concrets pour Jour 2
5. Estimer le temps passé par grande activité

Checklist qualité :

- Un lecteur externe peut reproduire vos tests avec vos notes
- Chaque découverte est sourcée (screenshot/log/commande)
- Les échecs sont documentés (pas que les succès !)
- Le vocabulaire est technique et précis
- Les prochaines étapes sont actionnables (pas de "analyser plus")

> **ERREUR FRÉQUENTE :** Ne pas documenter les échecs. En professionnel, un "ça n'a pas marché" DOIT être documenté avec le message d'erreur exact. Ça évite de refaire la même erreur J+3.

6. Préparation pour Jour 2

Checklist de Fin de Journée

Avant de partir, un consultant professionnel vérifie que :

- Environnement stable et sauvegardé (snapshot post-recon)
- Tous les artefacts collectés sont sauvegardés (hash, strings, screenshots)
- Daily log rédigé et committé (git)
- Plan pour J+1 clair (3-5 tâches prioritaires)
- Outils pour J+1 identifiés et pré-testés
- Aucune donnée sensible en clair sur machine hôte

6.1. Priorités par Projet pour Jour 2

Projet 1 - Malware : Priorités J2

1. Désassemblage complet dans Ghidra (toutes les fonctions nommées)
2. Analyse dynamique en sandbox (procmon + wireshark + regshot)
3. Extraction des strings obfusquées (script Python custom)
4. Début rédaction règle YARA basée sur patterns identifiés

Préparation : Tester Ghidra sur binaire simple, préparer template script Python de déobfuscation XOR.

Projet 2 - Web : Priorités J2

1. Audit manuel complet (toutes fonctionnalités testées)
2. Test des OWASP Top 10 (SQLi, XSS, CSRF, IDOR)
3. Identification de 5-10 vulnérabilités exploitables
4. Début exploitation (chaining pour impact maximal)

Préparation : Lire la doc OWASP Testing Guide v4, préparer checklist de tests.

Projet 3 - Mobile : Priorités J2

1. Analyse statique complète (code Java décompilé)
2. Mapping des endpoints API et authentification
3. Test de sécurité du stockage local (SharedPrefs, SQLite)
4. Premiers hooks Frida pour comprendre le runtime

Préparation : Lire doc Frida, préparer scripts de base (hook MainActivity, log network calls).

Lectures Recommandées pour Ce Soir

Projet 1 :

- Practical Malware Analysis - Chapitre 1-3 (révision)
- Blog de Malwarebytes sur TrickBot (derniers IOCs 2024)

Projet 2 :

- OWASP Testing Guide - Introduction et Méthodologie
- PortSwigger Academy - SQL Injection (refresh)

Projet 3 :

- OWASP MSTG (Mobile Security Testing Guide) - Chapitre Android Basics
- Frida Handbook - Getting Started + Android Examples

FIN JOUR 1 - Environnement Prêt, Cap sur l'Analyse Approfondie !

Récapitulatif des Compétences Jour 1

Compétence	Niveau Attendu	Validation
Setup environnement sécurisé	Maîtrise	VM opérationnelle + snapshot
Méthodologie projet pro	Compréhension	Plan d'attaque validé
Reconnaissance passive	Application	IOCs/vulns identifiés
Documentation technique	Application	Daily log complet